Global Responsibility-Weighted Kalman-Momentum Updates

1 Executive Summary

This proposal extends the winner-take-all prototype mutation to a global, responsibility-weighted memory update. For each eligible embedding, all active prototypes across all classes receive an update. The update magnitude is controlled by three independent factors:

1. **Global similarity responsibility:** Prototypes more similar to the current embedding receive a larger displacement.
2. **Per-prototype Kalman uncertainty:** Each prototype uses its own uncertainty and Kalman gain to determine how much of the proposed movement it accepts.
3. **Per-prototype momentum:** Each prototype maintains its own velocity state, smoothing its history of responsibility-weighted displacements.

Because global soft updating can cause multiple prototypes to collapse into the same region, the proposal adds a soft similarity penalty. This penalty operates on the matrix of normalized prototype retrieval keys and discourages pairwise similarities above a configurable threshold without requiring every prototype to be perfectly orthogonal.

The experiment therefore tests whether a memory that writes using the same soft competition used during retrieval can improve representation sharing, adaptation, and robustness while preserving prototype diversity.

2 Research Question and Hypothesis

The central question is:

> Does updating every prototype according to its global similarity responsibility, individual uncertainty, and individual momentum produce a better memory than updating only one selected prototype?

The hypothesis is that global responsibility-weighted updates will allow prototypes to share information from each observation. The most similar prototype should make the largest movement, while less similar prototypes make smaller movements. Across different observations, different prototypes may receive the largest responsibilities and develop complementary locations.

The diversity penalty is intended to prevent this sharing mechanism from degenerating into prototype collapse. It should separate prototypes only when their retrieval directions become excessively similar, while allowing useful within-class similarity below the configured threshold.

3 Network Architecture and Gradient Isolation

Let the encoder and direct classifier be:

$$f_\theta : \mathcal{X} \to \mathbb{R}^D, \qquad g_\phi : \mathbb{R}^D \to \mathbb{R}^C.$$

For an input x_i, the encoder produces:

$$z_i = f_\theta(x_i).$$

The direct classifier is trained using only the direct representation:

$$\mathcal{L}_{\text{CE}} = -\log \frac{\exp(g_\phi(z_i)_{y_i})}{\sum_{c=1}^{C} \exp(g_\phi(z_i)_c)}.$$

Prototype updates are detached from the computation graph. The memory does not send gradients through the encoder or classifier. This preserves the comparison between a direct CNN path and a non-differentiable associative memory path.

4 Memory Representation

The memory contains N prototype slots. Each prototype has:

$$M_j \in \mathbb{R}^D, \qquad V_j \in \mathbb{R}^D, \qquad P_j \in \mathbb{R}_{>0}.$$

where:

- M_j is the raw prototype value.

- V_j is the prototype-specific momentum velocity.
- P_j is the prototype-specific uncertainty.

Each prototype also has a class label c_j and an activity indicator a_j. The class label is retained for reporting and downstream classification, but it does not restrict the responsibility calculation or the update in this proposal.

The memory may be organized as K slots per class, giving:

$$N = CK.$$

The number of slots and their initial allocation are held constant across compared methods.

5 Warm-Up, Centering, and Initialization

During the warm-up period, the direct network trains without memory mutation. At the warm-up boundary, a global embedding center is computed:

$$\mu = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{x_i \in \mathcal{D}_{\text{train}}} f_\theta(x_i).$$

The center is frozen after it is computed. All retrieval keys, responsibilities, and diversity calculations use this same center:

$$\widetilde{z}_i = \frac{z_i - \mu}{\|z_i - \mu\|_2 + \epsilon_{\text{norm}}}.$$

$$\widetilde{M}_j = \frac{M_j - \mu}{\|M_j - \mu\|_2 + \epsilon_{\text{norm}}}.$$

The initial prototype values are constructed using the same initialization procedure for every compared method. Each prototype starts with:

$$V_j \leftarrow 0, \qquad P_j \leftarrow P_0.$$

This ensures that later differences are caused by the update rule rather than different initial memory geometry.

5.1 Optional Dynamic Global Centering

The frozen-center configuration is the primary setting. As an optional ablation, the global center may be updated after warm-up to track encoder representation drift. The center

remains global: it is computed over the current training embedding distribution, not separately per class or per prototype.

At an update boundary t, estimate the current embedding center with either a full training pass or a representative embedding sample:

$$\widehat{\mu}_t = \frac{1}{|\mathcal{B}_{\mu,t}|} \sum_{x_i \in \mathcal{B}_{\mu,t}} f_{\theta_t}(x_i).$$

The dynamic center can be updated with an exponential moving average:

$$\mu_t = \rho_\mu \mu_{t-1} + (1 - \rho_\mu)\widehat{\mu}_t,$$

where $\rho_\mu \in [0, 1)$ controls how quickly the reference follows the current embedding distribution. Alternatively, the center can be replaced periodically by $\widehat{\mu}_t$ without smoothing. The update frequency is a separate hyperparameter and may be once per epoch or once every T_μ epochs.

When dynamic centering is enabled, the same current center μ_t must be used for every query and prototype involved in a given write or read operation:

$$\widetilde{z}_{i,t} = \frac{z_{i,t} - \mu_t}{\|z_{i,t} - \mu_t\|_2 + \epsilon_{\text{norm}}}.$$

$$\widetilde{M}_{j,t} = \frac{M_{j,t} - \mu_t}{\|M_{j,t} - \mu_t\|_2 + \epsilon_{\text{norm}}}.$$

The raw prototype values M_j are not automatically translated when the center changes. The dynamic center changes the coordinate reference used for similarity, responsibility, and diversity calculations, while the memory values remain in the encoder's raw embedding space. This prevents a center update from silently adding a second prototype mutation.

Dynamic centering must be evaluated against the frozen-center baseline because it trades coordinate stability for better tracking of representation drift:

- **Frozen center:** stable reference and easier interpretation.
- **Periodic center:** follows later-epoch embedding means in discrete steps.
- **EMA center:** follows drift smoothly while reducing batch-to-batch noise.

The center update must occur at a defined boundary, such as the end of an epoch, rather than halfway through a minibatch. This ensures that all responsibilities and prototype updates within one operation use a consistent reference.

6 Accuracy-Guided Update Eligibility

The original accuracy-guided admission protocol is retained to avoid changing both the admission rule and the update rule at the same time.

For each training example, compare the direct and memory predictions:

$$\text{Category}(i) = \begin{cases} \text{Both Correct}, & \widehat{y}_i^{\text{direct}} = y_i \wedge \widehat{y}_i^{\text{memory}} = y_i, \\ \text{Harmed}, & \widehat{y}_i^{\text{direct}} = y_i \wedge \widehat{y}_i^{\text{memory}} \neq y_i, \\ \text{Helped}, & \widehat{y}_i^{\text{direct}} \neq y_i \wedge \widehat{y}_i^{\text{memory}} = y_i, \\ \text{Both Wrong}, & \widehat{y}_i^{\text{direct}} \neq y_i \wedge \widehat{y}_i^{\text{memory}} \neq y_i. \end{cases}$$

The primary experiment updates memory only for harmed examples:

$$\text{Eligible}(i) \iff \text{Category}(i) = \text{Harmed}.$$

This keeps directly misclassified or potentially unreliable embeddings out of the memory write path. An additional ablation may remove this gate and update on every post-warm-up example, but it must be reported separately.

7 Global Responsibility Calculation

For an eligible embedding z_i, calculate centered cosine similarity against every active prototype, regardless of class:

$$s_{ij} = \widetilde{z}_i^{\mathsf{T}} \widetilde{M}_j.$$

Let β_{write} be the write-temperature parameter. The global responsibility of prototype j is:

$$r_{ij} = \frac{\exp(\beta_{\text{write}} s_{ij})}{\sum_{k:a_k=1} \exp(\beta_{\text{write}} s_{ik})}.$$

Responsibilities satisfy:

$$r_{ij} \geq 0, \qquad \sum_{j:a_j=1} r_{ij} = 1.$$

A high responsibility means that prototype j is highly aligned with the current embedding. A low responsibility does not necessarily equal zero, so prototypes from other classes can receive small updates under this global mechanism.

The write temperature controls concentration:

- Large β_{write} approaches winner-take-all behavior.
- Small β_{write} distributes updates across more prototypes.

The write temperature is separate from the retrieval temperature so that write sharpness and read sharpness can be studied independently.

7.1 Optional Normal and Combined Similarity Ablation

The primary configuration uses centered cosine similarity. As an optional ablation, compare it with normal cosine similarity, which does not subtract the global center:

$$s_{ij}^{\text{normal}} = \frac{z_i^{\mathsf{T}} M_j}{\|z_i\|_2 \, \|M_j\|_2}.$$

The centered score is:

$$s_{ij}^{\text{centered}} = \frac{(z_i - \mu)^{\mathsf{T}} (M_j - \mu)}{\|z_i - \mu\|_2 \, \|M_j - \mu\|_2}.$$

An optional combined score can preserve both absolute direction and direction relative to the global center:

$$s_{ij}^{\text{combined}} = \alpha s_{ij}^{\text{normal}} + (1 - \alpha) s_{ij}^{\text{centered}},$$

where $\alpha = \alpha_{\text{mix}}$ and $0 \leq \alpha \leq 1$. The selected score replaces s_{ij} in the responsibility equation and, if the same retrieval mode is selected, in the read attention equation:

$$s_{ij} \in \left\{ s_{ij}^{\text{centered}}, s_{ij}^{\text{normal}}, s_{ij}^{\text{combined}} \right\}.$$

The optional comparison should evaluate:

1. Centered similarity only: $\alpha_{\text{mix}} = 0$ in the combined form.
2. Normal similarity only: $\alpha_{\text{mix}} = 1$ in the combined form.
3. Combined similarity: $0 < \alpha_{\text{mix}} < 1$.

The choice of similarity mode must be kept consistent between writing responsibilities and retrieval unless a separate read/write mismatch ablation is being studied. The value of α_{mix} must be selected using the validation split.

8 Responsibility-Weighted Kalman-Momentum Update

Let the current baseline accuracy be A_n. The observation noise is:

$$R_n = (1 - A_n) + \epsilon_{\text{stab}}.$$

Each prototype uses its own uncertainty to compute its Kalman gain:

$$K_j = \frac{P_j}{P_j + R_n}.$$

The responsibility-weighted innovation for prototype j is:

$$\delta_{ij} = r_{ij}(z_i - M_j).$$

The prototype-specific momentum state is updated as:

$$V_j \leftarrow \gamma V_j + (1 - \gamma)\delta_{ij}.$$

The Kalman-gated target movement is then:

$$\Delta_{ij}^{\text{KM}} = K_j V_j.$$

Before diversity correction, the prototype becomes:

$$M_j^{\text{temp}} \leftarrow M_j + \Delta_{ij}^{\text{KM}}.$$

Finally, uncertainty is propagated independently for every prototype:

$$P_j \leftarrow (1 - K_j)P_j + Q.$$

The complete update is applied to every active prototype. Therefore, the update strength is jointly determined by:

$$\text{Update Strength} \propto \text{Responsibility} \times \text{Momentum Direction} \times \text{Kalman Trust}.$$

This is not a class-restricted write. Class labels do not determine which prototypes update; global similarity does.

9 Soft Similarity Penalty Matrix

9.1 Retrieval-Key Gram Matrix

After the temporary Kalman-momentum update, construct centered normalized keys for all active prototypes and stack them row-wise:

$$\widetilde{M} = \begin{bmatrix} \widetilde{M}_1^\mathsf{T} \\ \widetilde{M}_2^\mathsf{T} \\ \vdots \\ \widetilde{M}_N^\mathsf{T} \end{bmatrix} \in \mathbb{R}^{N \times D}.$$

The prototype similarity matrix is:

$$G = \widetilde{M}\widetilde{M}^\mathsf{T}.$$

Each entry is:

$$G_{jk} = \widetilde{M}_j^\mathsf{T} \widetilde{M}_k.$$

The diagonal entries are approximately one and represent self-similarity.

9.2 Thresholded Soft Penalty

Let τ_{sim} be the maximum tolerated pairwise similarity and let $\mathcal{O}$ be the off-diagonal mask:

$$\mathcal{O}_{jk} = \begin{cases} 0, & j = k, \\ 1, & j \neq k. \end{cases}$$

Define the excess-similarity matrix:

$$E = \mathcal{O} \odot \max(0, G - \tau_{\text{sim}}),$$

where $\odot$ denotes elementwise multiplication. Only off-diagonal similarities above the threshold produce a penalty.

The soft diversity objective is:

$$\mathcal{L}_{\text{sim}} = \frac{1}{2} \|E\|_F^2 .$$

This does not require all prototypes to be orthogonal. It only discourages redundant

retrieval directions when their similarity exceeds τ_{sim}.

9.3 Matrix-Level Corrective Update

Let $\nabla_M \mathcal{L}_{\text{sim}}$ denote the gradient of this penalty with respect to the raw prototype matrix. The final prototype update is:

$$M \leftarrow M^{\text{temp}} - \lambda_{\text{sim}} \nabla_M \mathcal{L}_{\text{sim}}.$$

where $\lambda_{\text{sim}} \geq 0$ controls the strength of the diversity correction.

The penalty is applied after the responsibility-weighted Kalman-momentum step. It is a detached memory operation and does not backpropagate through the encoder. The diagonal of the similarity matrix is excluded so that a prototype is not pushed away from itself.

10 Retrieval

Retrieval uses all active prototypes and the same centered normalized keys:

$$w_{ij} = \frac{\exp\left(\beta_{\text{read}}\, \widetilde{z}_i^{\mathsf{T}} \widetilde{M}_j\right)}{\sum_{k:a_k=1} \exp\left(\beta_{\text{read}}\, \widetilde{z}_i^{\mathsf{T}} \widetilde{M}_k\right)}.$$

The retrieved representation uses raw prototype values:

$$h_i = \sum_{j:a_j=1} w_{ij} M_j.$$

Centering and normalization determine attention weights, while raw prototype values determine the retrieved content. The write temperature β_{write} and read temperature β_{read} may be equal in the primary configuration, but should remain separately configurable.

11 Controlled Comparisons

The following methods should use identical data, initialization, training schedule, seeds, and evaluation procedures:

1. **Winner-Take-All Update**

   Update only the selected prototype, matching the original Experiment 2 protocol.

2. **Global Responsibility-Weighted Kalman-Momentum**

   Update every active prototype using r_{ij}, its own P_j, and its own V_j, without the similarity penalty.

3. **Global Soft Update with Similarity Penalty**

   Use the global responsibility-weighted Kalman-momentum update followed by the thresholded matrix-level soft similarity correction. This is the proposed method.

4. **Global Soft Update with Uniform Uncertainty**

   Use global responsibilities but replace per-prototype uncertainty with a shared uncertainty value. This isolates the contribution of individual Kalman states.

5. **Global Soft Update with No Momentum**

   Set the velocity to the current responsibility-weighted innovation. This isolates the contribution of prototype-specific momentum.

12 Evaluation Metrics

Evaluate every method over independent random seeds.

12.1 Classification Performance

Record:

- Direct clean training, validation, and test accuracy.
- Memory clean training, validation, and test accuracy.
- Direct baseline accuracy at every corruption level.
- Memory accuracy at every corruption level.
- Helped, harmed, and net accuracy change.

12.2 Prototype Geometry

Record the mean and maximum off-diagonal similarity:

$$\overline{G}_{\text{off}} = \frac{1}{N(N-1)} \sum_{j \neq k} G_{jk}.$$

Also record:

- Number of pairs with $G_{jk} > \tau_{\text{sim}}$.
- Within-class and between-class similarity distributions.
- Prototype norm statistics.
- Prototype displacement per update.

12.3 Update Dynamics

Record:

- Responsibility entropy.
- Per-prototype update frequency.
- Per-prototype mean responsibility.
- Per-prototype Kalman gain.
- Per-prototype uncertainty.
- Per-prototype velocity norm.
- Cross-class update magnitude.

These measurements determine whether global updating is genuinely distributed or has effectively become winner-take-all.

13 Expected Failure Modes and Safeguards

13.1 Prototype Collapse

If many prototypes receive similar responsibilities for the same observations, they may converge to similar directions. Monitor G and increase λ_{sim} or reduce the responsibility temperature only through validation-controlled ablations.

13.2 Cross-Class Drift

Because every prototype updates, a prototype from an incorrect class can move toward an observation. Monitor cross-class update magnitudes and compare them with same-class update magnitudes.

13.3 Over-Separation

An overly strong similarity penalty may force useful same-class prototypes apart. The thresholded penalty and a small validated λ_{sim} are preferred over hard orthogonality.

13.4 Diffuse Responsibilities

If β_{write} is too small, almost every prototype receives a substantial update. This can make the method unstable and erase specialization.

13.5 Uncertainty Imbalance

Prototypes with persistently high uncertainty may dominate updates even when their responsibilities are modest. Report the joint distribution of r_{ij} and K_j rather than inspecting either quantity alone.

14 Hyperparameters

Table 1: Hyperparameters of the proposed method.

Symbol	Description
C	Number of target classes
D	Embedding dimension
N	Total number of prototype slots
K	Prototype slots per class, when capacity is class-organized
E_{warmup}	Warm-up duration before memory writes
β_{write}	Responsibility concentration temperature
β_{read}	Retrieval attention temperature
γ	Momentum retention coefficient
P_0	Initial prototype uncertainty
Q	Process-noise term
ϵ_{stab}	Measurement-noise stability constant
ϵ_{norm}	Normalization stability constant
τ_{sim}	Maximum tolerated pairwise key similarity
λ_{sim}	Soft similarity penalty strength
α_{mix}	Optional normal/centered similarity mixing weight
ρ_μ	Optional dynamic-center EMA retention coefficient
T_μ	Optional number of epochs between dynamic-center updates
A_n	Current baseline accuracy used for measurement noise

All hyperparameters affecting the proposed method must be selected using the validation split. The test split is used only for final reporting.

15 Expected Contribution

This experiment isolates whether a continuous Hopfield memory benefits from matching its soft read operation with a soft write operation. Unlike a winner-take-all update, the proposed method allows every prototype to respond to every eligible embedding, while preserving prototype-specific adaptation through individual uncertainty and momentum states.

The soft similarity penalty provides a matrix-level mechanism for retaining prototype diversity without imposing exact orthogonality. The resulting study will show whether global responsibility-weighted updates improve retrieval accuracy and corruption robustness, and whether those gains justify the risks of cross-class drift and increased update coupling.